

Identifying, Exploring, and Understanding the DRAM Communication Scheduling Space for DNN Accelerators

ABSTRACT

Modern Deep Neural Network (DNN) accelerators are equipped with increasingly larger on-chip buffers to provide more opportunities to optimize the increasingly severe DRAM bandwidth pressure. However, current research utilizing buffer optimizations for DRAM communication still primarily focuses on single-layer dataflow scheduling optimization. As buffers grow large enough to accommodate most single-layer weights in most networks, the impact of single-layer dataflow optimization on DRAM communication diminishes significantly. Therefore, developing new paradigms to fully leverage the increasingly abundant on-chip buffer resources to optimize the increasingly constrained DRAM bandwidth has become particularly important, which remains an open challenge.

To solve this challenge, we first identify the optimization opportunities in DRAM communication scheduling by analyzing the shortcomings of existing works on the layer fusion optimization paradigm and recognizing the vast potential of the prefetching-and-delayed-storing optimization paradigm based on two key observations. To fully exploit these optimization opportunities, we first develop an encoding method and a corresponding parsing method to represent different DRAM communication scheduling schemes and depict the overall space of DRAM communication scheduling. Then, to thoroughly explore the space of DRAM communication scheduling for diverse accelerators and workloads, we develop an end-to-end scheduling framework, Tensor Shuttle, which will be deployed on a commercial accelerator. Compared with the state-of-the-art (SOTA) Cocco framework, Tensor Shuttle can, on average, achieve a $\times$ performance improvement and a % reduction in energy cost simultaneously. Then we leverage Tensor Shuttle to perform design space exploration and analyze the DRAM communication scheduling space, achieving some interesting insights. Moreover, Tensor Shuttle will be open-sourced.

1. INTRODUCTION

In order to process a variety of tasks with better performance and accuracy, Deep Neural Networks (DNNs) are rapidly becoming more complex and heavy [7, 8, 11, 28, 33]. Accelerators with more computing units, larger buffers, and higher memory bandwidth have been developed to accelerate these DNN workloads [14, 16, 17, 21, 24, 27].

However, under modern semiconductor processes, the rate of increase in DRAM bandwidth lags behind the rate of transistor density growth, a longstanding issue [30]. This dispar-

ity is even more pronounced in DNN accelerators [30, 39], which are more specialized and have a larger proportion of dedicated compute elements than traditional chips like CPU. As a result, DRAM communication is increasingly becoming a performance bottleneck in DNN computation [30, 39]. To mitigate this bottleneck, accelerators are equipped with increasingly larger on-chip buffers [14, 16, 17, 21, 24, 27], providing opportunities to optimize DRAM communication by exploiting reuse opportunities in DNNs. However, how to fully leveraging these buffer resources to improve the performance and energy efficiency of DNN accelerators is still an open challenge.

Several works leverage buffer resources to reduce DRAM accesses under the paradigm of “layer fusion” [1, 20, 23, 26, 32, 39]. This approach involves buffering the feature maps (fmaps) produced by earlier layers on-chip, allowing subsequent consuming layers to directly read them. This strategy avoids the overhead of first writing back to DRAM and then reading from it, thereby reducing DRAM access costs. This optimization paradigm holds immense potential; for instance, Cocco achieved performance improvements ranging from 1.89% to 50.33% by merely exploring the option of which layers to fuse. Beyond this, there are numerous dimensions worth exploring under this paradigm, such as execution order and execution granularity. However, like Cocco, most existing studies [1, 26, 32] have only focused on a small portion of this optimization space. Therefore, we believe that the complex optimization dimensions within the layer fusion paradigm have not been clearly delineated or defined, much less thoroughly explored and understood in the context of the entire layer-fusion optimization space.

While reducing DRAM access is an important approach for optimizing DRAM communication, we have identified another previously overlooked approach—prefetching and delayed storing. We focus on this approach and believe it has potential based on two insightful observations: (1) With fine-grained layer fusion, much data can be reused on-chip. Traditional double buffer prefetching and data store strategies [3, 26, 32] (i.e., prefetching the data required for the next workload and storing the current workload’s output feature maps during the next workload) cannot fully utilize bandwidth, resulting in significant DRAM bandwidth idle time during intermediate tile computations (Fig. 2). (2) In modern DNN networks, the ratio of DRAM bandwidth demand to computation demand varies significantly across different layers. After layer fusion, the overall ratio of DRAM bandwidth demand to computation demand for different fused layer groups also varies greatly (as shown in Fig. 3). These

two observations indicate that with the application of layer fusion technology, DRAM bandwidth usage during the entire computation process becomes very uneven—sometimes leading to congestion due to high demand and sometimes causing bandwidth resource waste due to low demand. This motivates us to apply prefetching and delayed storing techniques to alleviate the uneven DRAM communication load. However, choosing the appropriate timing for prefetching and storing is a non-trivial problem. Clearly defining, thoroughly exploring, and understanding this issue is a significant challenge.

“Layer fusion” and “prefetching and delayed storing” each have their own optimization spaces, but they are not independent and are intricately coupled. This is reflected in the following points: 1) Both paradigms trade buffer usage for DRAM communication optimization, and 2) Layer fusion affects the types and quantities of data that need to communicate with DRAM. Therefore, we define the complex space formed by these two paradigms as the DRAM Communication Scheduling Space. From the previous introduction, it is evident that the individual spaces have not been clearly defined, let alone the definition, exploration, and understanding of this overall space.

After identifying the challenges and optimization potential of DRAM communication scheduling, we make the following contributions to address these challenges, exploit this potential, and enhance our understanding of DRAM communication scheduling.

We first introduce a comprehensive encoding method with two categories and a total of six attributes to encode the scheduling schemes in the DRAM Communication Scheduling Space, then we show how to parse each encode into actual hardware behaviours. Based on this encoding method, we define and illustrate the DRAM communication optimization space, and the complex trade-offs behind it. Existing works can all be described using our encoding method, representing merely a small subset within the space we have defined. To the best of our knowledge, this work is *the first* to comprehensively define and analyze the DRAM communication optimization space.

To explore the DRAM Communication Scheduling Space defined by the aforementioned encoding method, we develop an end-to-end framework, Tensor Shuttle, which is equipped with a two-stage exploration algorithm. We have successfully built a complete flow based on Tensor Shuttle, from model input to instruction generation, on a chip that is about to go into mass production.

Then, we conduct extensive experiments on different workloads, platforms, and batch sizes, demonstrating *** performance improvements compared with the state-of-the-art framework, Cocco. In addition, we use Tensor Shuttle to explore and analyze the architectural design space of the DRAM Communication Scheduling Space, gaining some interesting insights. For example, ***

2. HARDWARE BASIS

In this section, we introduce a generic large-scale DNN accelerator template adopted in this paper 1, which represents a significant portion of mainstream commercial DNN accelerators [2, 10, 15, 16, 17, 24, 27].

It primarily consists of DRAM, a Global Buffer, and sev-

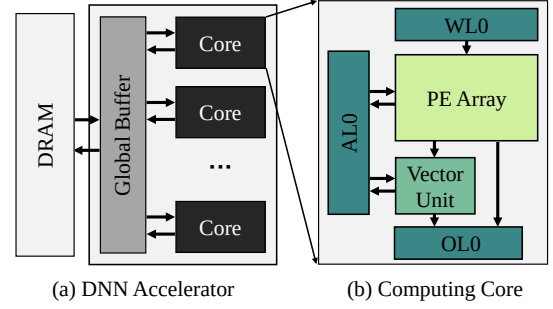


Figure 1: DNN Accelerator Template

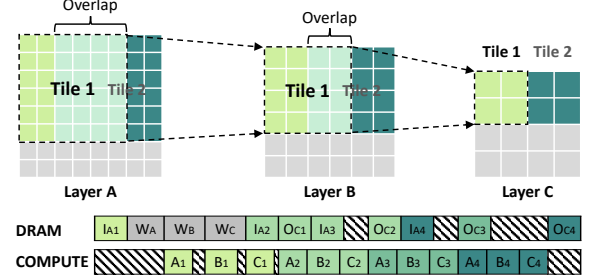


Figure 2: Layer Fusion Paradigm

eral cores. The Global Buffer is shared among all cores. Each core has a private small buffer/register files for rapid memory access by internal computing units; the PE array is used for computing GEMM/Conv operations, and the Vector Unit is designed for computing other vector/scalar operations, such as element-wise add, max pooling, etc.

3. MOTIVATION ANALYSIS

The Global Buffer plays a crucial role in optimizing DRAM communication within DNN accelerators, and research on this topic has been a hot topic since the advent of DNN accelerators. However, most current studies focus on optimizing dataflow for small-scale DNN accelerators, specifically on how to tile a single layer into small parts and adjust the computation order of these parts to optimize DRAM communication [4, 5, 9, 12, 13, 17, 18, 22, 25, 29, 31, 34, 36, 37].

As DNN accelerators have evolved, architects have equipped them with increasingly larger Global Buffers, with capacities reaching tens or even hundreds of megabytes [2, 6, 10, 15, 16, 21, 24]. Such large buffers can accommodate most layers of most networks, significantly reducing the effects of optimizing dataflow for DRAM communication. Therefore, developing new techniques to fully leverage the rapidly growing on-chip buffer resources to optimize the increasingly bottlenecked DRAM bandwidth is crucial and remains an open challenge. There are significantly fewer related articles compared to those focused on dataflow optimization for small-scale accelerators.

3.1 Layer Fusion

Layer fusion is an important paradigm for using buffers to optimize DRAM communication. Fig. 2 illustrates the fusion and computation method, where multiple fused layers are

partially computed in series, relying on on-chip buffers for switching fmaps. The optimization dimensions within this paradigm are numerous and complex, creating a vast optimization space. A straightforward optimization dimension is which layers to fuse, which affects both DRAM access savings and buffer occupancy. This is the primary focus of most existing studies [32, 38]. Another evident optimization space is the computation granularity of each fused layer group, i.e., the size of each part (Fig. ??). This affects buffer occupancy, halo overlap overhead, and the effectiveness of intra-core optimizations (detailed analysis in Sec. ??). DeFENIS [26] has analyzed this dimension. Most other studies address this issue using heuristic rules [32, 38], overlooking optimization opportunities within this space. These two dimensions are just simple examples; there are other optimization dimensions neglected by existing studies (introduced in Sec. 4.1.1). Even for these two dimensions, most existing work focuses on one while adopting heuristic strategies for the other [26, 32, 38].

figure 2-3 unbalance demand

3.2 Prefetching and Delayed Storing

Besides layer fusion, we have identified another optimization opportunity overlooked by existing literature—prefetching and delayed storing. We believe this optimization opportunity also has great potential based on the following two insightful observations.

The first observation is that, in terms of total volume, the ratio of DRAM access demand to compute demand varies greatly across different fused layer groups (as shown by the red and blue dots in Fig. ??). This results in significant differences in DRAM bandwidth requirements for different fused layer groups. The fundamental factor leading to this phenomenon is that in modern DNNs, the DRAM access demand and compute demand vary greatly among different layers. As shown in Fig. 2, the layer with the highest DRAM access/computation time ratio in Transformer and ResNet-50 is 2.7 times and 5 times that of the layer with the lowest ratio, respectively.

The second observation is that, from a micro perspective, the existing prefetching and storing strategy (double buffer strategy) leads to uneven DRAM bandwidth usage within a fused layer group in the layer fusion scenario. Specifically, as shown in Fig. 2, the first tile of most layers might experience congestion due to prefetching weights, while subsequent tiles are likely to be idle because the weights are already on-chip and many feature maps can be reused on-chip.

Combining these two observations, we find that controlling the timing of data prefetching and storing to utilize idle DRAM bandwidth and alleviate peak-time pressure has great potential. However, achieving precise control over this process is non-trivial. Clearly defining, thoroughly exploring, and understanding this issue is a significant challenge.

3.3 Combine Them Together

“Layer fusion” and “prefetching and delayed storing” are two optimization paradigms with complex interrelationships, which lies in two aspects. The first aspect lies in the fact that both optimization paradigms inherently trade buffer usage for DRAM communication optimization, leading to competition for buffer resources between the two. Additionally, layer

fusion affects the tensors that need DRAM communication, which in turn impacts the optimization space for prefetching and delayed storing. As introduced before, these interdependent optimization spaces have yet to be clearly defined individually, let alone the overall optimization space they constitute. This motivates us to clearly define the overall optimization space, thoroughly explore it, and thereby enhance our understanding of this space.

4. DELINEATION AND ANALYSIS OF DRAM COMMUNICATION OPTIMIZATION SPACE

4.1 Encoding Format and Parsing Methods

In this section, we will introduce the semantics of our encoding method, how it translates into actual hardware behaviors, and the trade-offs behind different encoding choices through a practical example of a five-layer network (A to E) shown in Fig. 3.

As shown in Fig. 3, the encoding format has six attributes, which can be divided into two categories: Layer-Fusion-related Attributes (LFA) and DRAM-Load-and-store-related Attributes (DLSA). Consequently, the overall parsing process is also divided into two stages. The first stage involves parsing the LFA to determine: 1) the computation granularity and order, as well as the buffer occupancy of tensors that are reused on-chip, and 2) all tiles that need to interact with DRAM. The second stage involves parsing the DLSA to derive the specific timing and buffer occupancy details for all tensors that need to be loaded/stored to/from DRAM.

4.1.1 Layer-Fusion-related Attributes

LFA includes **Layer Order**, **Fine-grained Layer-fusion Cut (FLC)**, **Tile Number**, and **DRAM Cut (DC)**, which will be introduced one by one.

The first attribute, Layer Order, arranges these layers into a serial sequence, representing their coarse-grained execution order. Hence, a valid layer order cannot have any dependency that goes from right to left, as this would result in a scenario where the data needed by a layer calculated earlier is not yet computed. Its format is a vector. In Fig. 3 is an example of an order; swapping the order of *D* and *E* would still result in a valid order, but swapping *A* and *B* would not be valid.

The second attribute, Fine-grained Layer-fusion Cut (FLC), records the cut locations, which divides layers into Fine-grained Layer-fusion Groups (FLG). All FLCs constitute the FLC set (FLCS). Each FLG possesses a **Tile Number attribute**, which determines the computing granularity of the FLG. Layers within an FLG are processed continuously at the granularity of tiles rather than completely finishing the first layer before starting the second, thereby saving on-chip buffer overhead. For the example in the left top of Fig. 3, the layers are cut into three FLGs (*A, B, C, E, D*) with Tile Number 1, 2, 2, respectively. Within the FLG (*C, D, E*), each layer is segmented into two tiles which are then processed in sequence. We use a heuristic strategy to allocate the tile number’s division dimensions to each layer. This strategy prioritizes splitting the batch, then the height and width of the output feature maps (keeping them as equal as possible to reduce overlap). The reason for not splitting the channel dimension is that splitting the channel would prevent the next

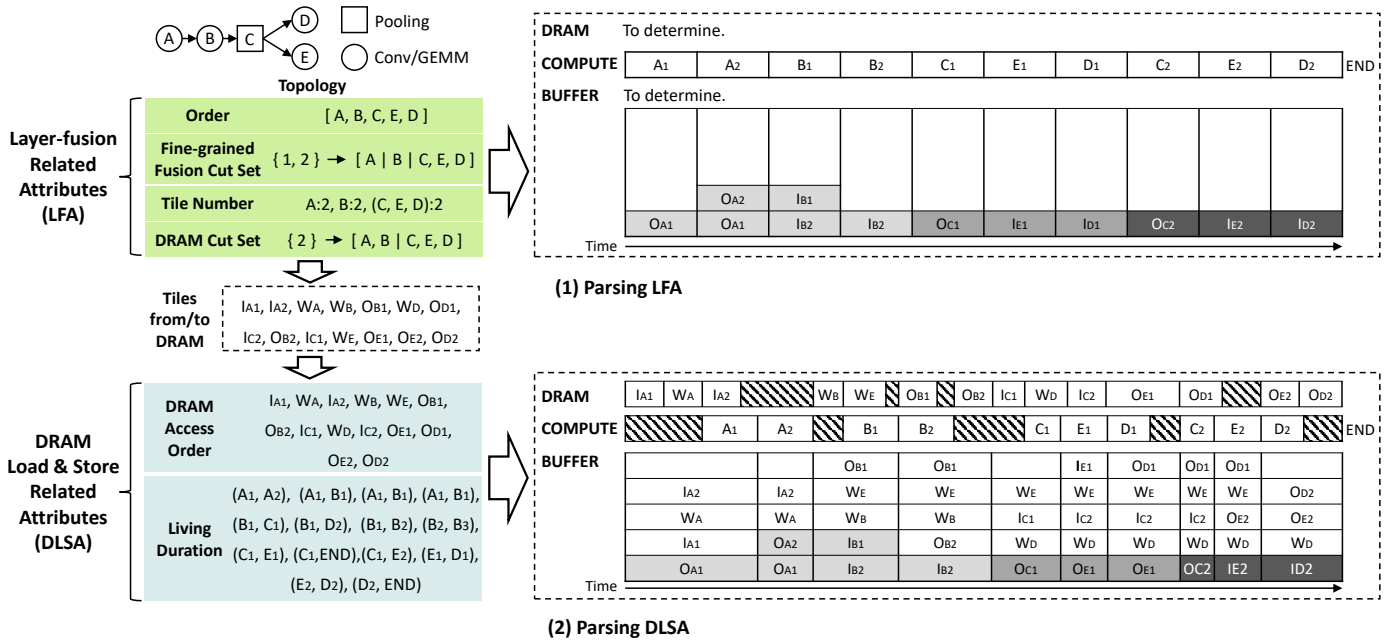


Figure 3: Parsing an Example Encode of five layers into Actual Scheduling Schemes. The DRAM, COMPUTE, and BUFFER in the right part show the DRAM access, workload computation, and buffer usage, respectively. We use M_i to represent the i th tile of layer M , and $I/W/O_{M(i)}$ to represent the Ifmaps, Weights, and Ofmaps of $M(i)$. In the BUFFER, blocks with the same background color represent the same data.

layer from accessing all channels, making it impossible to fuse multiple layers. For example, in Fig. 2, with a batch size of 1 and a tile number of 4, we split the height and width dimensions each by 2 (the channel dimension is not shown). It is worth emphasizing that when the intermediate layers involve operations such as convolution or pooling, which produce overlaps, the size of each tile may be larger than $1/\text{Tile}$ of the size of the fmaps (e.g., Layer A and B in Fig. 2). The specific method for determining each tile size of intermediate layers has already been proposed in Cocco [32] and DeFENIS [26], so we directly adopt their strategy. The trade-off involved with the Tile Number primarily concerns the balance between buffer usage, overlap overhead, and computational efficiency within the core. Finer-grained tiles imply reduced buffer usage, but if they involve layers that produce overlaps, they may incur greater computational and memory access overheads. Additionally, the computational efficiency within the core may decrease (as smaller tiles imply a reduced scheduling space).

By parsing the above three attributes, we can derive the entire computation sequence (the COMPUTE row in Fig. 3).

DRAM Cut records the cut locations to divide the layers into Layer-fusion Groups (LGs), with dependencies between different LGs needing to be first sent to DRAM and then retrieved for computation. All DRAM Cuts constitute the DRAM Cut Set (DCS). For example, B has a dependency on C , and there is a DRAM cut between B and C . Therefore, the Ofmaps of B needs to be sent to DRAM, and C needs to retrieve the Ifmaps from DRAM. Thus, All the requirements for each tile's interaction with DRAM can be determined as follows: if a layer has weights, then its weight requirements

are loaded from DRAM. For example, except for C which does not have weights, the weights of all other layers need to be loaded from DRAM (W_M in the left part of Fig. 3). If a layer has forward dependencies that span different LGs (or its input is the overall network input), then the ifmaps of its related tiles need to be loaded from DRAM (I_{M_i} in the left part of Fig. 3). If it has backward dependencies that span different DCGs (or is the overall network output), then the ofmaps of its related tiles need to be written back to DRAM (O_{M_i} in the left part of Fig. 3). The trade-off involved in DCS primarily concerns the balance between buffer requirements and the volume of DRAM access. Generally speaking, the more fused layers there are (the fewer the DCS), the lower the number of DRAM accesses will be, but the demand for buffer capacity will increase.

The remaining data dependencies that do not cross a DRAM cut can be directly reused on-chip. The buffer occupancy duration for such data extends from the production of the Ofmaps tile to the consumption of the tile. For example, if A 's Ofmaps are consumed by two tiles of B , its buffer occupancy is shown in Fig. 3(1). In Fig. 3(1), we only show the buffer allocation for on-chip data transfers, while all data related to communication with DRAM are not displayed (to be determined in the DRAM Load & Store Phase).

Since the DCS is a subset of the FLCS, this means that each DCG contains one or more FLGs. Within a DCG, each FLG is a fine-grained multi-layer fusion unit. If there are data dependencies between different FLGs, the producing FLG needs to aggregate the Ofmaps from different tiles before they can be used by the consuming FLG. Once the computation for an FLG is completed, its weights can be released and

no longer occupy the buffer. For example in Fig. 3, the weights of A can be released after its two tiles are computed. B needs to wait until all Ofmaps of A are computed before it can start its computation. Additionally, the Ofmaps of A (which serve as the Ifmaps for B) need to be stored on-chip and consumed progressively. Combining the above analysis, it can be seen that the FLG primarily involves a trade-off in buffer occupation. FLG can free up some of the buffer space occupied by weights, but it will cause the fmaps to accumulate and be consumed gradually.

4.1.2 DRAM-Load-and-Store-related Attributes

DLSA includes **Tensor Order and Living Duration**, which will be introduced one by one.

The **Tensor Order** attribute determines the access sequence for all tensors that need to communicate with DRAM, such as the tensors in the dashed squares of Fig. 3.

Every tensor that requires communication with DRAM has a adjustable **Living Duration** attribute, which is a 2-element tuple (start,end). Both start and end are tile IDs. This attribute has dual implications: First, within this time frame, buffer is allocated to this tensor. For example in Fig. 3, the Living Duration of W_E is (B_1, D_2) . Therefore, it is allocated buffer space for storage within Tiles B_1 and D_2 . Second, it schedules the timing for loading and storing. Specifically, the start indicates when the tensor can begin to be loaded/stored, and the end marks when the tensor must complete transmission; otherwise, subsequent compute tasks cannot start. For example, the *end* of O_{E_1} is C_2 , so if its store has not completed, C_2 cannot start. However, the actual start time is more complex; loading/storing data can begin at the start tile ID, but the specific loading time depends on whether the required data is ready and whether the loads and stores preceding this tensor in the Tensor Order have been completed. For example, the *Start* of I_{C_2} is C_1 , but the loading of the preceding W_D takes a long time, causing the actual load of I_{C_2} to begin only in the middle of E_1 .

4.2 Space Size Comparison

From the preceding discussion, we can infer that in our notation, each scheme is encoded by six variable attributes. Thus, our notation constructs an approximately six-dimensional vast optimization space. In contrast, if we map the scheduling from the state-of-the-art (SOTA) Cocco [32] into our notation, only Layer Order and DRAM Cut are variable (with FLC identical with DRAM Cut), and the scheduling corresponding to the other four attributes is determined by heuristic strategies, making its explorable optimization space much smaller than ours. To bridge the gap and fully explore this vast optimization space, uncovering its potential for optimization, we introduce our Two-stage optimization framework, Tensor Shuttle, in the following chapter.

5. TENSOR SHUTTLE FRAMEWORK

5.1 Tensor Shuttle Overview

As shown in Fig. 4, Tensor Shuttle is an end-to-end DNN scheduling framework. Tensor Shuttle takes as inputs: (1) a DNN model description file produced by high-level frameworks like PyTorch, which is further processed by the Model

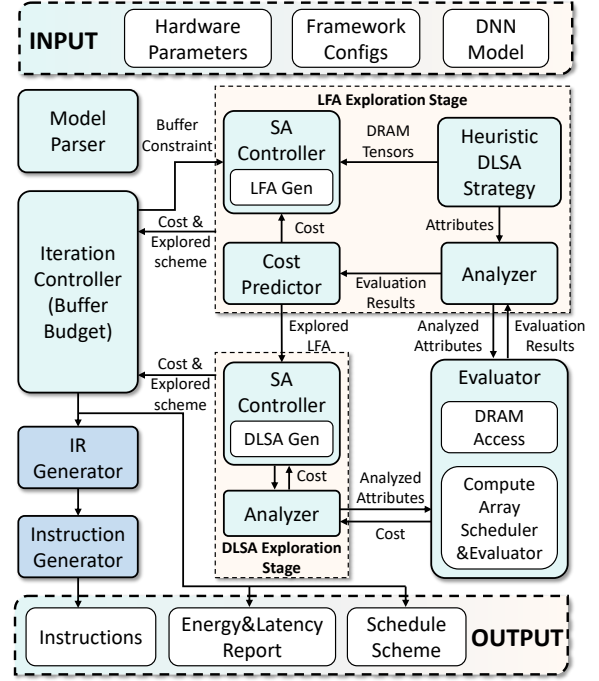


Figure 4: Tensor Shuttle Framework

Parser to extract the necessary information for scheduling; (2) hardware configuration (computing power, DRAM bandwidth, buffer size, and so on); (3) framework settings (optimization goal, searching hyperparameters, and so on). After scheduling, Tensor Shuttle outputs: (1) energy cost and performance reports; (2) a detailed scheduling scheme; (3) instructions.

The optimization objective of Tensor Shuttle is configurable as $Energy^n \times Delay^m$, where n and m are adjustable parameters that prioritize energy efficiency and performance according to different needs. Energy and latency refer to the total energy consumption and delay for processing a batch of samples. Therefore, to evaluate the effectiveness of Tensor Shuttle in latency-centric scenarios, small batch sizes can be used, while larger batch sizes can be employed for throughput-centric scenarios.

As shown in Fig. 4, the main search process employs a two-stage approach, separately varying and searching LFA and DLSA in each stage, with each stage using a simulated annealing algorithm for independent searches. The reason for using a two-stage search is that, as analyzed in Sec. 4.1, changes in LFA significantly impact DRAM communication requirements, making it difficult to retain good solutions obtained by varying DLSA attributes while continuously varying LFA. To better consider the mutual influence of the two stages, we equip the LFA Exploration Stage with a Cost Predictor. This predictor can correct the schemes of the first stage, which use basic prefetch and store strategies, by predicting the capabilities of the second stage and providing the corrected cost. This allows the first stage to take into account the second stage’s capabilities, avoiding the cost of optimizing DRAM access requirements in the first stage that could be optimized in the second stage, while also providing better optimization

potential for the second stage (for example, executing FLGs with ample DRAM bandwidth and buffer space before and after an FLG with severe DRAM bottleneck).

Outside this two-stage search, there is an Iteration Controller, which dynamically allocates buffer budgets to both competing stages based on the buffer usage status and optimization effects of the best solutions found in the previous iteration. This ensures that the optimization potential of both stages is fully realized, preventing one stage from having excessive buffer resources while the other is insufficient. Specifically, each iteration involves a complete exploration of the two-stage DRAM communication scheduling space.

The optimal results can be directly outputted from the Buffer Controller as scheduling schemes and reports, or they can be converted into a more easily parsable intermediate representation (IR) through our IR Generator module. This IR can then be input into the Instruction Generator module to produce actual instructions. We have already built and successfully run such a compilation flow for our in-house commercial accelerator.

5.2 Evaluator

The default Evaluator is based on the general hardware template introduced in Sec. 2. It can evaluate the generated scheduling schemes for different networks under various hardware parameters, assessing their energy cost and performance. As shown in Fig. 4, its input includes an ordered sequence of compute tile tasks, where each compute tile contains information about its layer, dimensions, dependencies, etc. Additionally, there is an ordered sequence of DRAM communication requests, where each request includes (start, end), dependencies, type (ifmaps, weights, or ofmaps), dimensions, etc.

The evaluation of overall energy consumption and performance follows a local-to-global approach. First, each compute tile and DRAM load/store request is evaluated individually, and then the overall assessment is conducted.

For each computing tile, the Core Array Scheduler & Evaluator performs scheduling search and power & performance evaluation, selecting the optimal scheduling scheme for the core array. The corresponding energy consumption and computing time of the searched optimal scheme are considered the tile's energy consumption and computing time. There has been extensive research on mapping a tile to a core array [12, 13, 19, 22, 25, 29, 34, 36, 37], so we use a classic maestro-like [22] search-based method for the scheduling exploration. The evaluator also adopts a maestro-like approach [22, 29]. Specifically, the energy consumption calculation involves counting the read/write operations of each level of the buffer, the number of various computations, and the data transmission volume on various interconnects, then multiplying each by the corresponding unit energy consumption and summing them up. Performance evaluation is conducted using an event-driven approach.

For each DRAM communication tensor, its energy consumption is calculated by multiplying the read and write data volumes by their respective unit energy consumption for DRAM reads and writes, and then summing them up. The read and write time is calculated as data volume divided by bandwidth.

The overall total energy consumption is calculated by summing up the energy consumption of the above subcomponents. The total computation time is derived based on the evaluated times of all computing tiles and DRAM communication tensors, using the following rule.

For each DRAM communication requirement, it can start execution only when the following three conditions are met: 1) The preceding requirement has been completed, 2) Its start time is greater than or equal to the current tile ID, 3) If the request is for ofmaps, it must wait until the generating computing tile has finished. For example, in Fig. 3, W_B has a start of B_1 . Although the previous request has already been completed, it must still wait until A_2 finishes before it can begin. Similarly, although I_{C_2} 's start is C_1 , the preceding request is not completed until E_1 , so it can only start at E_1 .

For each computing tile task, it can start execution only when the following two conditions are met: 1) All required data (ifmaps, weights, parameters, etc.) are available. For example, A_1 , B_1 , and C_1 cannot follow the preceding computation tasks immediately because the required data are not available at the end of the preceding task. 2) The end of the preceding tile's DRAM communication request must be completed. For instance, C_2 cannot follow D_1 because O_{E_1} 's end is D_1 , and D_1 cannot start until O_{E_1} has finished execution.

5.3 Exploration Stage

The exploration of the entire DRAM communication scheduling space is divided into two stages, with each stage using a standard simulated annealing algorithm. Specifically, in each iteration, the SA controller selects an operator to transform the encode with a certain probability and then analyzes and evaluates the transformed scheme. If the overall cost is lower, the change is accepted; otherwise, it is accepted with a probability that decreases as the number of iterations increases. Next, we will introduce the specific SA operators and strategies for the two stages.

5.4 LFA Exploration Stage

In the first exploration stage (LFA Exploration Stage), the SA operators primarily transform the LFA, while the DLSA is determined using a heuristic strategy (as introduced in Sec. 3.3). The specific operators are introduced below:

Change Layer Order: Randomly select a layer that has another layer with which it can exchange order, and swap their orders.

Change Tile Number: Randomly select an FLG and either multiply or divide its Tile Number by 2.

Add/Delete An FLC: Randomly select a position with or without a cut to add or remove an FLC. Specifically, adding an FLC results in two FLGs, each inheriting the tile number attribute from the original FLG. Removing an FLC merges two FLGs into one, with the new FLG's tile number inherited probabilistically based on the layer count ratio of the original two FLGs.

Add/Delete An DC: Select a position within an FLCS that is or is not a DC, and add or remove a DC.

5.5 DLSA Exploration Stage

5.6 Validation

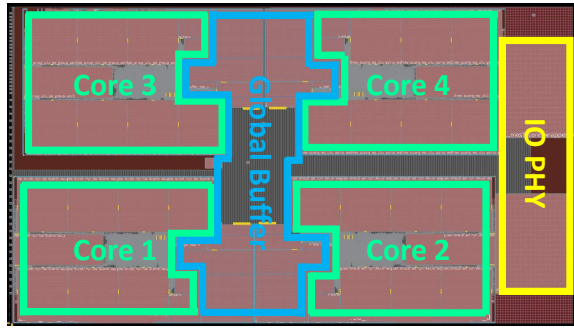


Figure 5: Layout of A Commercial Accelerator Using Tensor Shuttle

We validate the evaluator of Tensor Shuttle with the results from the Synopsys ZEBUServer4 Emulation System, which is the verification platform for our chip currently in tape-out (Fig. 5). Using ResNet50 [11] and Transformer [33], the average error is 5.65%.

5.7 Generality and Portability of Tensor Shuttle

The proposed encoding and Tensor Shuttle framework exhibit excellent versatility for two main reasons: 1. The template depicted in Figure 1 is highly general, encompassing many accelerators from both the industry and academia [16, 17, 35] add some citation; 2. The hardware behavior information encoded by the encoding is very general (i.e., the size of computation, loading, and storing, as well as instruction dependencies dispatch timing).

Our Tensor Shuttle also possesses excellent portability, which is due not only to the aforementioned reasons but also because our framework is designed with a robust modular architecture. Different accelerators may have distinct core micro-architectures, so it is only necessary to replace the corresponding Core Array Scheduler and Evaluator modules, as well as the instruction generation module within the Evaluator, to adapt to various accelerators. We have developed a comprehensive flow for our accelerator (Fig. 5), which serves as an exemplary guide for the process of porting to other accelerators.

6. EVALUATION

6.1 Experiments Setup

REFERENCES

- [1] M. Alwani, H. Chen, M. Ferdman, and P. A. Milder, “Fused-layer CNN accelerators,” in *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 2016, pp. 22:1–22:12. [Online]. Available: <https://doi.org/10.1109/MICRO.2016.7783725>
- [2] P. Bannon, G. Venkataramanan, D. D. Sarma, and E. Talpes, “Computer and redundancy solution for the full self-driving computer,” in *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 2019, pp. 1–22. [Online]. Available: <https://doi.org/10.1109/HOTCHIPS.2019.8875645>
- [3] J. Cai, Y. Wei, Z. Wu, S. Peng, and K. Ma, “Inter-layer scheduling space definition and exploration for tiled accelerators,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, Y. Solihin and M. A. Heinrich, Eds. ACM, 2023, pp. 13:1–13:17. [Online]. Available: <https://doi.org/10.1145/3579371.3589048>
- [4] Y. Chen, J. S. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” in *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 2016, pp. 367–379. [Online]. Available: <https://doi.org/10.1109/ISCA.2016.40>
- [5] Y. Chen, T. Yang, J. S. Emer, and V. Sze, “Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices,” *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, 2019. [Online]. Available: <https://doi.org/10.1109/JETCAS.2019.2910232>
- [6] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, “Dadiannao: A machine-learning supercomputer,” in *47th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2014, Cambridge, United Kingdom, December 13-17, 2014*. IEEE Computer Society, 2014, pp. 609–622. [Online]. Available: <https://doi.org/10.1109/MICRO.2014.58>
- [7] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: <https://doi.org/10.18653/v1/n19-1423>
- [8] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2021.
- [9] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, “Shidiannao: shifting vision processing closer to the sensor,” in *Proceedings of the 42nd Annual International Symposium on Computer Architecture, Portland, OR, USA, June 13-17, 2015*, D. T. Marr and D. H. Albonesi, Eds. ACM, 2015, pp. 92–104. [Online]. Available: <https://doi.org/10.1145/2749469.2750389>
- [10] A. Firoozshahian, J. Coburn, R. Levenstein, R. Nattoji, A. Kamath, O. Wu, G. Grewal, H. Aepala, B. Jakka, B. Dreyer, A. Hutchin, U. Diril, K. Nair, E. K. Aredestani, M. Schatz, Y. Hao, R. Komuravelli, K. Ho, S. Abu Asa, J. Shajrawi, K. Quinn, N. Sreedhara, P. Kansal, W. Wei, D. Jayaraman, L. Cheng, P. Chopda, E. Wang, A. Bikumandla, A. Karthik Sengottuvel, K. Thottampudi, A. Narasimha, B. Dodds, C. Gao, J. Zhang, M. Al-Sanabani, A. Zehtabioskuie, J. Fix, H. Yu, R. Li, K. Gondkar, J. Montgomery, M. Tsai, S. Dwarakapuram, S. Desai, N. Avidan, P. Ramani, K. Narayanan, A. Mathews, S. Gopal, M. Naumov, V. Rao, K. Noru, H. Reddy, P. Venkatapuram, and A. Bjorlin, “Mtia: First generation silicon targeting meta’s recommendation systems,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ser. ISCA ’23*. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589348>
- [11] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>
- [12] K. Hegde, P. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, “Mind mappings: enabling efficient algorithm-accelerator mapping space search,” in *ASPLOS ’21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021*, T. Sherwood, E. D. Berger, and C. Kozyrakis, Eds. ACM, 2021, pp. 943–958. [Online]. Available: <https://doi.org/10.1145/3445814.3446762>
- [13] Q. Huang, A. Kalaiah, M. Kang, J. Demmel, G. Dinh, J. Wawrzynek, T. Norell, and Y. S. Shao, “Cosa: Scheduling by constrained optimization for spatial accelerators,” in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 554–566. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00050>
- [14] M. James, M. Tom, P. Groeneveld, and V. Kibardin, “ISPD 2020 physical mapping of neural networks on a wafer-scale deep learning accelerator,” in *ISPD 2020: International Symposium on Physical*

- Design, Taipei, Taiwan, March 29 - April 1, 2020, delayed to September 20-23, 2020*, W. Swartz and J. Lienig, Eds. ACM, 2020, pp. 145–149. [Online]. Available: <https://doi.org/10.1145/3372780.3380846>
- [15] Y. Jiao, L. Han, R. Jin, Y. Su, C. Ho, L. Yin, Y. Li, L. Chen, Z. Chen, L. Liu, Z. He, Y. Yan, J. He, J. Mao, X. Zai, X. Wu, Y. Zhou, M. Gu, G. Zhu, R. Zhong, W. Lee, P. Chen, Y. Chen, W. Li, D. Xiao, Q. Yan, M. Zhuang, J. Chen, Y. Tian, Y. Lin, W. Wu, H. Li, and Z. Dou, “A 12nm programmable convolution-efficient neural-processing-unit chip achieving 825tops,” in *2020 IEEE International Solid-State Circuits Conference, ISSCC 2020, San Francisco, CA, USA, February 16-20, 2020*. IEEE, 2020, pp. 136–140. [Online]. Available: <https://doi.org/10.1109/ISSCC19947.2020.9062984>
 - [16] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. C. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. A. Patterson, “Ten lessons from three generations shaped google’s tpuv4i: Industrial product,” in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 1–14. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00010>
 - [17] N. P. Jouppi, C. Young, N. Patil, D. A. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017*. ACM, 2017, pp. 1–12. [Online]. Available: <https://doi.org/10.1145/3079856.3080246>
 - [18] S. Kao, G. Jeong, and T. Krishna, “Confucius: Autonomous hardware resource assignment for DNN accelerators using reinforcement learning,” in *53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020, Athens, Greece, October 17-21, 2020*. IEEE, 2020, pp. 622–636. [Online]. Available: <https://doi.org/10.1109/MICRO50266.2020.00058>
 - [19] S. Kao and T. Krishna, “GAMMA: automating the HW mapping of DNN models on accelerators via genetic algorithm,” in *IEEE/ACM International Conference On Computer Aided Design, ICCAD 2020, San Diego, CA, USA, November 2-5, 2020*. IEEE, 2020, pp. 44:1–44:9. [Online]. Available: <https://doi.org/10.1145/3400302.3415639>
 - [20] S.-C. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, “Flat: An optimized dataflow for mitigating attention bottlenecks,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, 2023*, pp. 295–310.
 - [21] S. Knowles, “Graphcore,” in *IEEE Hot Chips 33 Symposium, HCS 2021, Palo Alto, CA, USA, August 22-24, 2021*. IEEE, 2021, pp. 1–25. [Online]. Available: <https://doi.org/10.1109/HCS52781.2021.9567075>
 - [22] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, “Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12-16, 2019*. ACM, 2019, pp. 754–768. [Online]. Available: <https://doi.org/10.1145/3352460.3358252>
 - [23] J. Lee, D. Shin, J. Lee, J. Lee, S. Kang, and H. Yoo, “A full HD 60 fps CNN super resolution processor with selective caching based layer fusion for mobile devices,” in *2019 IEEE Symposium on VLSI Circuits, Kyoto, Japan, June 9-14, 2019*. IEEE, 2019, p. 302. [Online]. Available: <https://doi.org/10.23919/VLSIC.2019.8778104>
 - [24] H. Liao, J. Tu, J. Xia, and X. Zhou, “Davinci: A scalable architecture for neural network computing,” in *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 2019, pp. 1–44. [Online]. Available: <https://doi.org/10.1109/HOTCHIPS.2019.8875654>
 - [25] L. Lu, N. Guan, Y. Wang, L. Jia, Z. Luo, J. Yin, J. Cong, and Y. Liang, “TENET: A framework for modeling tensor dataflow based on relation-centric notation,” in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 720–733. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00062>
 - [26] L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst, “Defines: Enabling fast exploration of the depth-first scheduling space for dnn accelerators through analytical modeling,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 570–583.
 - [27] T. Norrie, N. Patil, D. H. Yoon, G. Kurian, S. Li, J. Laudon, C. Young, N. P. Jouppi, and D. A. Patterson, “Google’s training chips revealed: Tpuv2 and tpuv3,” in *IEEE Hot Chips 32 Symposium, HCS 2020, Palo Alto, CA, USA, August 16-18, 2020*. IEEE, 2020, pp. 1–70. [Online]. Available: <https://doi.org/10.1109/HCS49909.2020.9220735>
 - [28] OpenAI, “Gpt-4 technical report,” 2023.
 - [29] A. Parashar, P. Raina, Y. S. Shao, Y. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. S. Emer, “Timeloop: A systematic approach to DNN accelerator evaluation,” in *IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2019, Madison, WI, USA, March 24-26, 2019*. IEEE, 2019, pp. 304–315. [Online]. Available: <https://doi.org/10.1109/ISPASS.2019.00042>
 - [30] Z. Sun, S. Kvatinsky, X. Si, A. Mehonic, Y. Cai, and R. Huang, “A full spectrum of computing-in-memory technologies,” *Nature Electronics*, pp. 1–13, 2023.
 - [31] Z. Tan, H. Cai, R. Dong, and K. Ma, “Nn-baton: DNN workload orchestration and chiplet granularity exploration for multichip accelerators,” in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 1013–1026. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00083>
 - [32] Z. Tan, Z. Zhu, and K. Ma, “Cocco: Hardware-mapping co-exploration towards memory capacity-communication optimization,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ser. ASPLOS ’24*. New York, NY, USA: Association for Computing Machinery, 2024, p. 69–84. [Online]. Available: <https://doi.org/10.1145/3617232.3624865>
 - [33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems, ser. NIPS’17*. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.
 - [34] R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. R. Pinckney, P. Raina, Y. Zhang, B. Zimmer, W. J. Dally, J. S. Emer, S. W. Keckler, and B. Khailany, “Magnet: A modular accelerator generator for neural networks,” in *Proceedings of the International Conference on Computer-Aided Design, ICCAD 2019, Westminster, CO, USA, November 4-7, 2019*, D. Z. Pan, Ed. ACM, 2019, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/ICCAD45719.2019.8942127>
 - [35] O. Wechsler, M. Behar, and B. Daga, “Spring hill (NNP-I 1000) intel’s data center inference chip,” in *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 2019, pp. 1–12. [Online]. Available: <https://doi.org/10.1109/HOTCHIPS.2019.8875671>
 - [36] Q. Xiao, S. Zheng, B. Wu, P. Xu, X. Qian, and Y. Liang, “HASCO: towards agile hardware and software co-design for tensor computation,” in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 1055–1068. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00086>
 - [37] X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina, C. Kozyrakis, and M. Horowitz, “Interstellar: Using halide’s scheduling language to analyze DNN accelerators,” in *ASPLOS ’20: Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16-20, 2020*, J. R. Larus, L. Ceze, and K. Strauss, Eds. ACM, 2020, pp. 369–383. [Online]. Available: <https://doi.org/10.1145/3373376.3378514>
 - [38] S. Zheng, X. Zhang, D. Ou, S. Tang, L. Liu, S. Wei, and S. Yin, “Efficient scheduling of irregular network structures on cnn

- accelerators,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 3408–3419, 2020.
- [39] S. Zheng, S. Chen, P. Song, R. Chen, X. Li, S. Yan, D. Lin, J. Leng, and Y. Liang, “Chimera: An analytical optimizing framework for effective compute-intensive operators fusion,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 1113–1126.